

Real-Time Active Defense & MTD Honeypot Lab

Documentation & Practical Lab Verification Report



Performed By:	Abdul Rafay
Target Environment:	Ubuntu server(rafayserver)
Platform:	Kali Linux, Ubuntu Server
Lab Title:	Real-Time Active Defense & MTD Honeypot Lab
Tool Used:	THC-Hydra, RockYou Wordlist, Kali Linux, Ubuntu Server, OpenSSH, iptables, SQLite, Python 3.

Table of Contents

1. Executive Summary & Overview	3
○ 1.1 Summary	3
○ 1.2 Proposed Solution	3
○ 1.3 Key Architecture & Features	3
2. System Architecture & Module Specifications	4
○ 2.1 Architecture Data Flow & Inter-Module Communication	4
○ 2.2 Core Module Breakdown	4
▪ 2.2.1 <code>main.py</code> (Core System Controller)	4
▪ 2.2.2 <code>coordinator.py</code> (MTD Shifting Engine)	4
▪ 2.2.3 <code>honeypot_server.py</code> (Decoy Services Listener)	5
▪ 2.2.4 <code>detector.py</code> (Threat Engine)	5
▪ 2.2.5 <code>firewall.py</code> (Active IP Mitigation Engine)	5
▪ 2.2.6 <code>database.py</code> (SQLite / Event State Storage)	5
▪ 2.2.7 <code>log_watcher.py</code> (Real-Time Log Parser)	5
▪ 2.2.8 <code>config.py</code> (System Configurations)	5
▪ 2.2.9 <code>admin.py</code> (CLI Management Tool)	6
3. Moving Target Defense (MTD) Mechanics	6
○ 3.1 Core Operational Principles	6
○ 3.2 Current Scope & Boundaries	7
4. Administrative Management Engine (<code>admin.py</code>)	8
○ 4.1 Supported Administrative Commands	8
○ 4.2 CLI Command Execution Proofs & Inspection	9
5. Practical Lab Verification & Testing (Proof of Concept)	10
○ 5.1 Test Environment Parameters	10
○ 5.2 Attack Tooling & Wordlist Configuration	10
○ 5.3 Step-by-Step Test Execution Lifecycle	11
▪ 5.3.1 Phase 1: High-Velocity Attack Initiation	11
▪ 5.3.2 Phase 2: Real-Time Detection & Automated Firewall Blocking	12
▪ 5.3.3 Phase 3: Administrative Inspection & Threat Audit	12
▪ 5.3.4 Phase 4: Administrative Remediation & System Restoration	13
6. Evaluation Matrix & Key Findings	14
7. Open Source Availability & Custom Web Deployment Guide	15
○ 7.1 Repository Access & Open-Source Footprint	15
○ 7.2 Step-by-Step Deployment Guide	16

1. Summary

The conventional approaches to network security have a strong dependence on static settings, predictable service ports, and constant response signatures. Thus, a static approach creates an asymmetrical advantage for hackers. It provides an opportunity to perform reconnaissance, network mapping, and automated high-speed brute force attacks on a regular basis without being detected or faced with any serious difficulties.

- **Proposed Solution**

In order to eliminate these shortcomings, the Dynamic Moving Target Defense (MTD) Honeypot Framework offers a deceptive and adaptive defense method. Through the constant changing of decoys' surface parameters, service response signatures, and listening settings, the framework keeps the attack surface constantly moving. Such dynamic movement makes the intelligence obtained by the attacker ineffective.

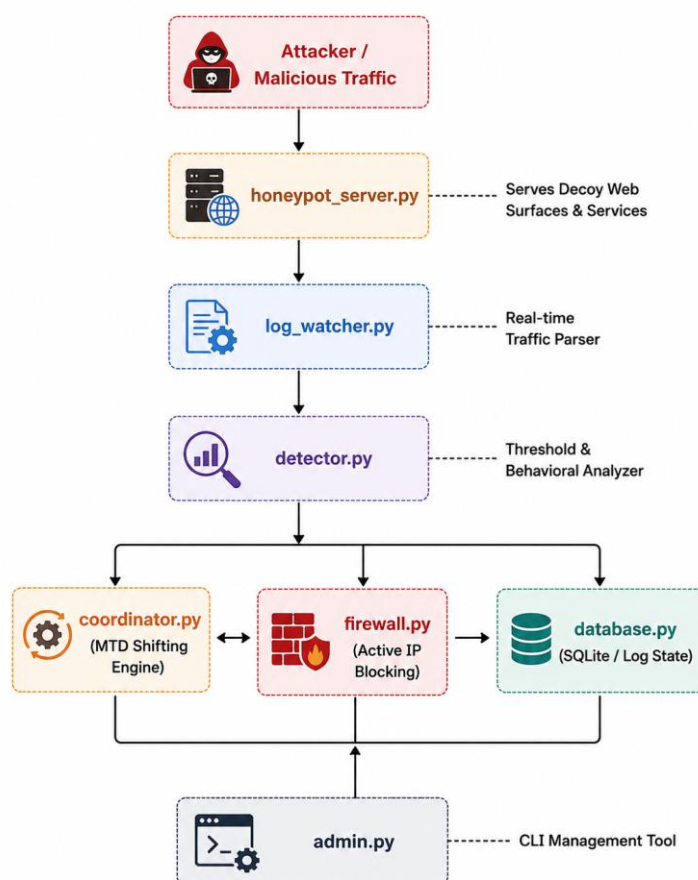
- **Architecture & Features**

- Asymmetric Invalidation of Reconnaissance: Threshold-triggered honeypot rotation prevents attackers from building a stable, consistent profile of the target service.
- Transport-Layer Isolation via Automated Detection: Built-in threat detection algorithms detect malicious connection velocity and signatures on Layer 4 (TCP/UDP).
- Dynamic Firewall Rules Generation: System automatically sends dynamic iptables/firewall rules to block IPs with malicious activities instantly.
- Log Monitoring & Database Logging: Real-time log monitoring and database logging record all the intruder behavior and payload signatures.

2. System Architecture & Module Specifications

The architecture is built on a clean decoupled Python architecture running in the local host environment (~/.hds). Every part serves a particular purpose in the chain of operations, starting from attack detection to its execution and then logging.

Architecture Data Flow & Inter-Module Communication

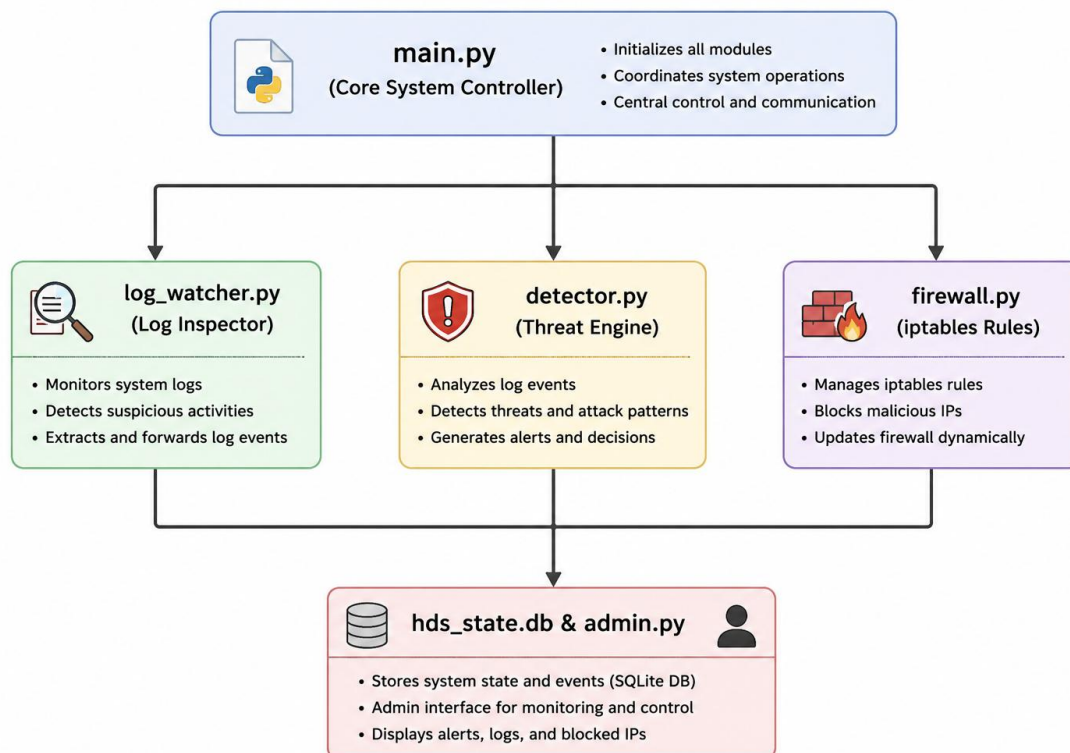


Module Breakdown

1. main.py: Acts as the central control which initiates background processes, monitors real-time logs, initializes MTD coordinator and binds the service listeners.

2. coordinator.py: Decision engine that redirects an attacker from the real service to a honeypot after a configurable number of failed attempts, and rotates them to a different honeypot after further failures on the same one — using iptables DNAT so the shift is transparent to the attacker.

3. **honeypot_server.py**: Implements decoy interfaces such as SSH listener and web application decoys that collect attacker credentials, payloads, and request details without actually utilizing system resources.
4. **detector.py**: Detects the traffic metrics according to pre-defined security rules. It records request rate, login attempts rate per IP and target username to detect the dictionary attacks on the spot.
5. **firewall.py**: Mitigation engine that communicates with system packet filtering mechanisms such as iptables and drops the incoming packets from the offending IP addresses.
6. **database.py**: Deals with the data storage of blocked IP addresses, targeted account names, reasons for violation string messages and time stamp UNIX values.
7. **log_watcher.py**: Log parser with low latency to stream current service logs, with minimal or no lag between any attack attempt and its analysis.
8. **config.py**: Configuration module containing security parameters, such as maximum number of login attempts per minute, blocking period and network interface configurations.
9. **admin.py**: Administrative command-line interface for checking system status, auditing active blocks and unblocking IPs or usernames manually.

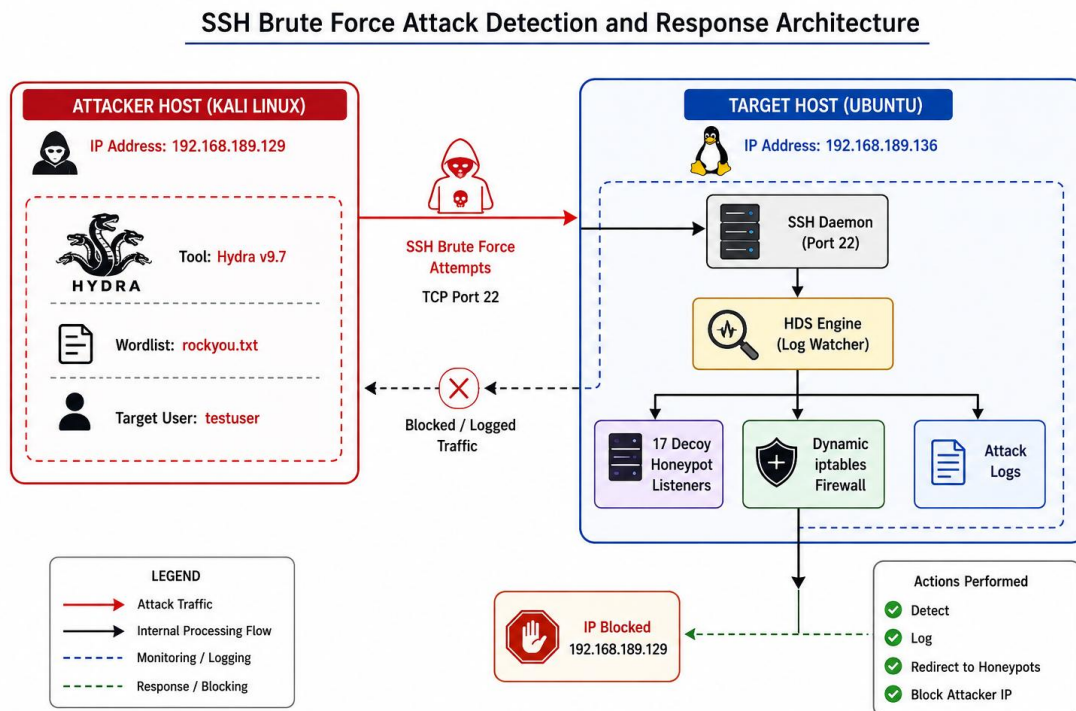


3. Moving Target Defense (MTD) Mechanics

Principle of Moving Target Defense is that attackers should always be faced with changing attack surfaces, thus adding to their cost and complexity while retaining minimum overheads for the defender.

Core Operational Principles

- **Asymmetric Cost of Attack:** The hackers incur great computational overhead, proxy lists, and wasted time analyzing dynamic or trap-based targets.
- **100 Percent Malicious Target:** There is no way that legitimate users will ever have a need to communicate with the honeypot endpoint; therefore, every communication can be categorized with certainty as malicious.
- **Outdated Reconnaissance Data:** Tools such as Nmap, Nikto, or Python scripts compile data that gets outdated within minutes through hopping parameters and ports.



Current Scope & Boundaries

In its current implementation, HDS operates entirely at the network/protocol level: all 17 decoys are TCP socket listeners impersonating services such as FTP, Telnet, SMTP, MySQL, RDP, and others. The framework does not currently include a web-based (HTML form) honeypot layer. A plug-and-play web frontend for decoy login, registration, and password-reset pages is listed as a future roadmap item in Section 7.

4. Administrative Management Engine (admin.py)

The admin.py utility gives system administrators full operational control over active threat blocklists and firewall states. It supports structured CLI commands for listing, unblocking, and manually managing network restrictions.

Supported Administrative Commands

Operation	Command Syntax	Functional Description
List Blocked Entities	<code>python3 admin.py list</code>	Displays all currently banned IP addresses and usernames along with trigger reasons and UNIX timestamps.
Unblock Target IP	<code>python3 admin.py unblock-ip <IP></code>	Removes the specified IP address from the database blocklist and flushes corresponding firewall drop rules.
Unblock Targeted User	<code>python3 admin.py unblock-username <USER></code>	Clears an account username restriction from the active threat database.
Manual IP Block	<code>python3 admin.py block-ip <IP></code>	Instantly imposes a firewall drop rule and database block on a specific IP address.
Manual Username Block	<code>python3 admin.py block-username <USER></code>	Places a username on the restriction list to block associated login vectors.

➤ First command(to view list):

```
rafay@rafayserver:~/hds$ sudo python3 admin.py list

=== Blocked IPs ===
192.168.189.129    reason: exceeded total attempt threshold at: 1785777638.4586086

=== Blocked Usernames ===
testuser          reason: used by malicious IP 192.168.189.129 at: 1785777638.4974248

rafay@rafayserver:~/hds$ |
```

➤ Second and third command (to unblock IP and Username):

```
rafay@rafayserver:~/hds$ sudo python3 admin.py unblock-ip 192.168.189.129
IP 192.168.189.129 unblocked.
rafay@rafayserver:~/hds$ sudo python3 admin.py unblock-username testuser
Username 'testuser' unblocked.
```

➤ Checking Whether unblocked or not:

```
rafay@rafayserver:~/hds$ sudo python3 admin.py list

=== Blocked IPs ===
(none)

=== Blocked Usernames ===
(none)

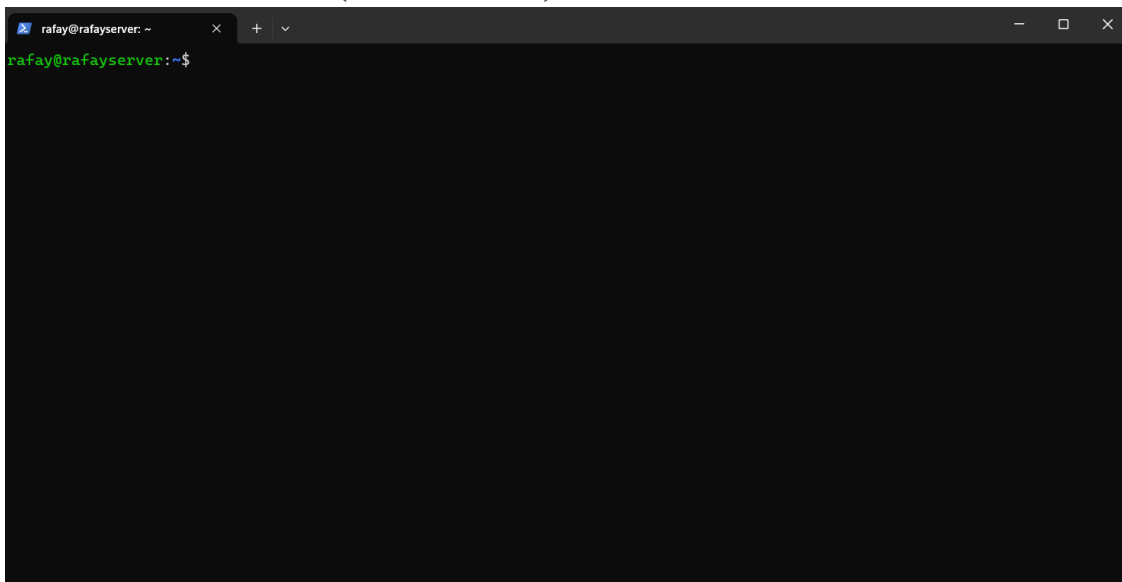
rafay@rafayserver:~/hds$ s|
```

5. Practical Lab Verification & Testing (Proof of Concept)

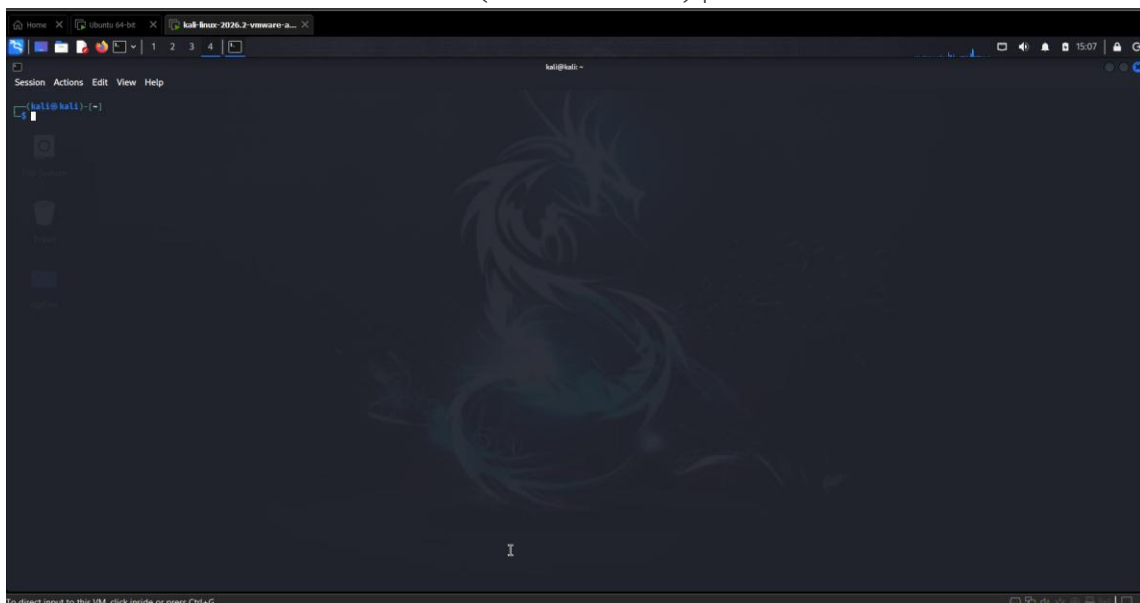
To evaluate the effectiveness of the detection and active defense mechanisms under realistic conditions, an automated SSH brute-force attack was conducted inside a isolated virtualization lab environment.

Test Environment Parameters

- **Target Environment:** Ubuntu Linux Server (Hostname: rafayserver) | Directory: ~/hds | IP Address: 192.168.189.136 (Interface ens33)



- **Attacker Environment:** Kali Linux (Hostname: kali) | IP Address: 192.168.189.129



- **Attack Tooling:** THC-Hydra v9.7 Automated Login Auditor

```
(kali@kali)-[~]
$ hydra -l testuser -P /usr/share/wordlists/rockyou.txt ssh://192.168.189.136
Hydra v9.7 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes (this is non-binding, these ** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-08-03 13:20:29
[WARNING] Many SSH configurations limit the number of parallel tasks, it is recommended to reduce the tasks: use -t 4
[DATA] max 16 tasks per 1 server, overall 16 tasks, 14344399 login tries (l:1/p:14344399), ~896525 tries per task
[DATA] attacking ssh://192.168.189.136:22/
```

- **Payload Wordlist:** /usr/share/wordlists/rockyou.txt (Wordlist parsing test targeting account 'testuser')

```
Applications
Session Actions Edit View Help

(kali@kali)-[~]
$ sudo gunzip /usr/share/wordlists/rockyou.txt.gz
```

Step-by-Step Test Execution Lifecycle

Phase 1: High-Velocity Attack Initiation

From the Kali Linux terminal (192.168.189.129), an automated multi-threaded SSH dictionary attack was launched against the target server:

```
hydra -l testuser -P /usr/share/wordlists/rockyou.txt ssh://192.168.189.136
```

```
(kali@kali)-[~]
$ hydra -l testuser -P /usr/share/wordlists/rockyou.txt ssh://192.168.189.136
Hydra v9.7 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes (this is non-binding, these ** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-08-03 13:20:29
[WARNING] Many SSH configurations limit the number of parallel tasks, it is recommended to reduce the tasks: use -t 4
[DATA] max 16 tasks per 1 server, overall 16 tasks, 14344399 login tries (l:1/p:14344399), ~896525 tries per task
[DATA] attacking ssh://192.168.189.136:22/
```

The attack tool established initial TCP handshake connections and initiated parallel authentication tasks:

```
Hydra v9.7 starting at 2026-08-03 13:20:29
[DATA] max 16 tasks per 1 server, overall 16 tasks, 14344399 login tries
[DATA] attacking ssh://192.168.189.136:22/
```

Phase 2: Real-Time Detection & Automated Firewall Blocking

As Hydra transmitted high-velocity login attempts, detector.py registered a threshold breach within seconds. Firewall rules were dynamically applied to drop incoming packets from 192.168.189.129.

The active attack was instantly severed mid-execution, forcing Hydra into a socket connection timeout:

```
[ERROR] could not connect to ssh://192.168.189.136:22 - Timeout connecting to 192.168.189.136
```

```
[DATA] attacking ssh://192.168.189.136:22/  
[ERROR] could not connect to ssh://192.168.189.136:22 - Timeout connecting to 192.168.189.136  
  
(kali㉿kali)-[~]  
$
```

Attack Neutralization Result: *The connection timeout proves that the active mitigation engine dropped attacker packets at the transport layer, effectively halting wordlist traversal and protecting host resources.*

Phase 3: Administrative Inspection & Threat Audit

On the target server (rafayserver), the administrative utility was executed to verify that the offending IP and targeted account were recorded:

```
sudo python3 admin.py list
```

```
rafay@rafayserver:~/hds$ sudo python3 admin.py list  
  
=== Blocked IPs ===  
192.168.189.129    reason: exceeded total attempt threshold at: 1785777638.4586086  
  
=== Blocked Usernames ===  
testuser         reason: used by malicious IP 192.168.189.129 at: 1785777638.4974248  
  
rafay@rafayserver:~/hds$ |
```

The system returned active block entries with exact trigger reasons and UNIX timestamps:

```
=== Blocked IPs ===  
192.168.189.129  reason: exceeded total attempt threshold at: 1785777638.4586086  
  
=== Blocked Usernames ===  
testuser        reason: used by malicious IP 192.168.189.129 at: 1785777638.4974248
```

Phase 4: Administrative Remediation & System Restoration

To test the unblocking workflow, the administrator executed unblock commands for both the restricted IP address and username:

```
sudo python3 admin.py unblock-ip 192.168.189.129
sudo python3 admin.py unblock-username testuser
```

```
rafay@rafayserver:~/hds$ sudo python3 admin.py unblock-ip 192.168.189.129
IP 192.168.189.129 unblocked.
rafay@rafayserver:~/hds$ sudo python3 admin.py unblock-username testuser
Username 'testuser' unblocked.
```

System confirmation output:

```
IP 192.168.189.129 unblocked.
Username 'testuser' unblocked.
```

A post-remediation verification query confirmed that all blocklists were successfully cleared:

```
sudo python3 admin.py list
```

```
rafay@rafayserver:~/hds$ sudo python3 admin.py list

=== Blocked IPs ===
  (none)

=== Blocked Usernames ===
  (none)

rafay@rafayserver:~/hds$ s|
```

```
=== Blocked IPs ===
  (none)

=== Blocked Usernames ===
  (none)
```

6. Evaluation Matrix & Key Findings

The empirical data collected through lab testing shows that the Moving Target Defense Honeypot Framework is effective in meeting all the target security objectives.

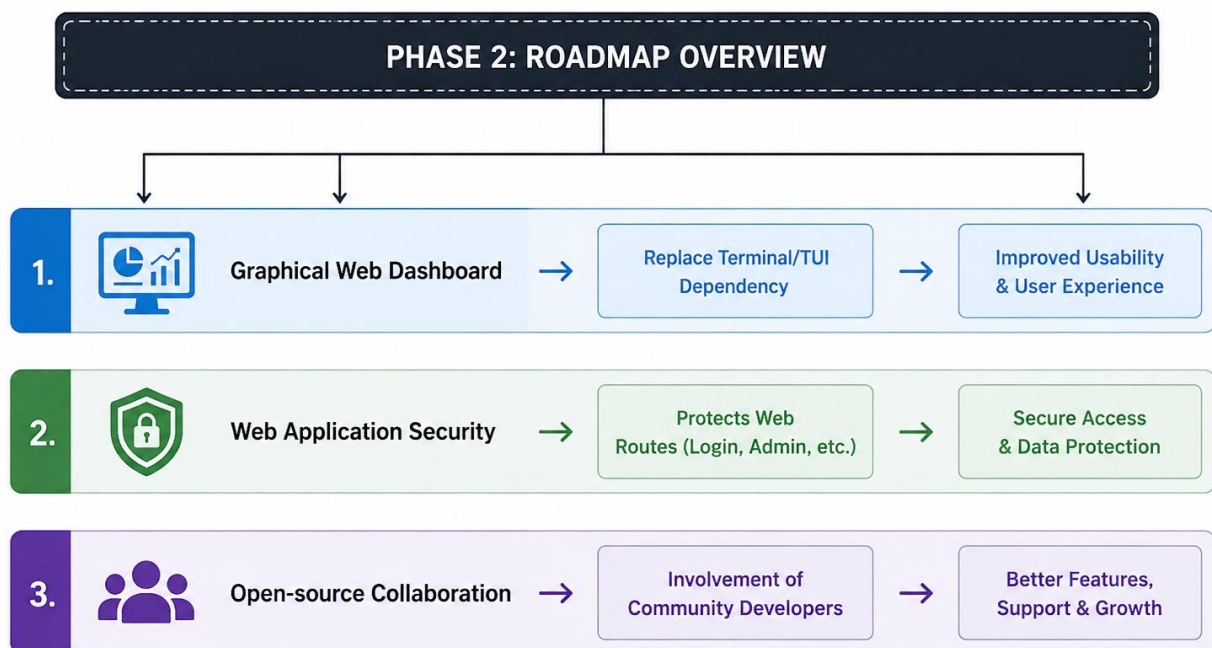
Test Phase / Metric	Target Security Objective	Observed System Behavior	Status
Reconnaissance Defense	Invalidate automated mapping and static signatures	Attacker silently redirected/rotated across decoy ports via MTD coordinator	PASS
Active Mitigation	Detect and terminate high-velocity brute-force attacks	Attacker IP 192.168.189.129 blocked; Hydra connection timed out	PASS
Threat Persistence	Log offending IPs, usernames, and timestamps	Database recorded violation reason and exact UNIX timestamp (1785777638)	PASS
Admin Management	Provide granular control to list and unblock entities	admin.py CLI listed active blocks and successfully cleared IP and username entries	PASS

7. Open Source Availability & Custom Web Deployment Guide

To encourage community auditing, collaborative security research, and seamless adoption across diverse web applications, this framework is published as an open-source project hosted on GitHub.

Repository Access & Open-Source Footprint

The entire codebase including the core engine, MTD coordinator, detection modules, and administrative utilities is available on GitHub. Security teams and developers can clone the public repository, inspect the architecture, review configuration options, and deploy the framework in local or cloud environments.



Step-by-Step Deployment Guide

To deploy HDS on a new Ubuntu server, follow these steps:

- **Step 1: Clone Repository:** Clone the public repository from GitHub to the target Linux server and review README.md for dependencies and system requirements.
- **Step 2: Install Dependencies:** Install Python 3 and iptables (sudo apt install python3 iptables). Place all project files in a single working directory (e.g., ~/hds).
- **Step 3: Configure config.py:** Set the real service log path and port, and tune the fail-count thresholds to the desired sensitivity.
- **Step 4: Extend Honeypot Coverage (optional):** Add new ports/service names to HONEYPOT_PORTS in config.py to cover additional protocols beyond the default 17.
- **Step 5: Launch:** Run sudo python3 main.py to start the honeypot listeners, log watcher, and MTD coordinator.

Note: A plug-and-play web-frontend layer (decoy login, register, and password-reset pages) is planned as a future enhancement — see the roadmap below — and is not part of the current release.